

05_c0_baseline_relocation

```
source("R/00_setup.R")
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
model_coords <- readRDS("model_coords.rds")
coords        <- readRDS("country_wave_coords.rds")

latest_gt <- coords %>%
  transmute(country = as.character(iso), wave = as.numeric(wave), gt1 = Dim1, gt2 = Dim2) %>%
  group_by(country) %>% filter(wave == max(wave)) %>% ungroup() %>%
  select(country, gt1, gt2)

#' ## The C0 point
#'
#' One coordinate per model. Where it lands is itself descriptive: a
#' mid-secular, high-self-expression position is not a neutral origin.

c0 <- model_coords %>%
  filter(condition == "C0") %>%
  group_by(model) %>%
  summarise(c0_1 = mean(Dim1, na.rm = TRUE), c0_2 = mean(Dim2, na.rm = TRUE), .groups = "drop")
cat("\n=== C0 coordinate per model (country-agnostic) ===\n"); print(c0)
```

```
=== C0 coordinate per model (country-agnostic) ===
```

```
# A tibble: 4 x 3
  model          c0_1 c0_2
  <chr>          <dbl> <dbl>
1 Qwen/Qwen3.6-Plus 0.372 0.397
2 claude-opus-4-8   0.0752 0.285
3 gemini-3.6-flash  0.182 0.342
4 gpt-5.5           0.189 0.373
```

```
c1 <- model_coords %>%
  filter(condition == "C1") %>%
  transmute(model, country = as.character(country), c1_1 = Dim1, c1_2 = Dim2)

#' Three quantities per model × country: how far the country-blind answer sits
#' from that country's truth, how far the country-named answer sits from it, and
#' how far naming the country moved the answer.

reloc <- c1 %>%
  inner_join(c0, by = "model") %>%
  inner_join(latest_gt, by = "country") %>%
  mutate(base_dist = sqrt((c0_1 - gt1)^2 + (c0_2 - gt2)^2),
         c1_dist = sqrt((c1_1 - gt1)^2 + (c1_2 - gt2)^2),
         reloc_dist = sqrt((c1_1 - c0_1)^2 + (c1_2 - c0_2)^2),
         improve = base_dist - c1_dist)

cat("\n=== point estimates ===\n")
```

```
=== point estimates ===
```

```
reloc %>% group_by(model) %>%
  summarise(n = n(), mean_base = round(mean(base_dist), 3),
            mean_c1 = round(mean(c1_dist), 3), mean_reloc = round(mean(reloc_dist), 3),
            .groups = "drop") %>% print()
```

```
# A tibble: 4 x 5
  model          n mean_base mean_c1 mean_reloc
  <chr>        <int>    <dbl>    <dbl>    <dbl>
1 Qwen/Qwen3.6-Plus 40    0.616    0.238    0.453
```

2	claude-opus-4-8	40	0.493	0.177	0.39
3	gemini-3.6-flash	40	0.532	0.153	0.545
4	gpt-5.5	40	0.55	0.193	0.486

```

b_base <- boot_paired(reloc, "base_dist")
b_reloc <- boot_paired(reloc, "reloc_dist")
b_impr <- boot_paired(reloc, "improve")

report_ci(b_base, "C0 baseline distance to truth, 95% CI")

```

=== C0 baseline distance to truth, 95% CI ===

	X2.5.	X50.	X97.5.	vs_ref
Qwen/Qwen3.6-Plus	0.533	0.615	0.701	ABOVE
claude-opus-4-8	0.433	0.492	0.556	ABOVE
gemini-3.6-flash	0.466	0.532	0.601	ABOVE
gpt-5.5	0.481	0.549	0.621	ABOVE

```

report_ci(b_reloc, "relocation, distance C0 to C1, 95% CI")

```

=== relocation, distance C0 to C1, 95% CI ===

	X2.5.	X50.	X97.5.	vs_ref
Qwen/Qwen3.6-Plus	0.372	0.452	0.536	ABOVE
claude-opus-4-8	0.333	0.389	0.445	ABOVE
gemini-3.6-flash	0.469	0.544	0.622	ABOVE
gpt-5.5	0.409	0.484	0.565	ABOVE

```

#' The improvement interval is the one that supports the claim that naming the
#' country makes the answer *better*. The relocation interval cannot: distance is
#' non-negative, so an interval excluding zero there is guaranteed and tests
#' nothing.

```

```

report_ci(b_impr, "accuracy gain from naming the country (C0 dist - C1 dist), 95% CI")

```

=== accuracy gain from naming the country (C0 dist - C1 dist), 95% CI ===

	X2.5.	X50.	X97.5.	vs_ref
Qwen/Qwen3.6-Plus	0.296	0.378	0.463	ABOVE
claude-opus-4-8	0.256	0.316	0.378	ABOVE
gemini-3.6-flash	0.307	0.379	0.457	ABOVE
gpt-5.5	0.284	0.357	0.432	ABOVE

```
#' ## Which societies does the default answer already resemble?
#'
```

```
#' Ranked within each model, since the C0 point differs by model.
```

```
ranked <- reloc %>%
  select(model, country, reloc_dist) %>%
  group_by(model) %>%
  mutate(rank_least_moved = rank(reloc_dist, ties.method = "min")) %>%
  ungroup()

cat("\n=== five least-moved countries per model ===\n")
```

```
=== five least-moved countries per model ===
```

```
ranked %>% filter(rank_least_moved <= 5) %>% arrange(model, rank_least_moved) %>%
  mutate(reloc_dist = round(reloc_dist, 3)) %>% as.data.frame() %>% print(row.names = FALSE)
```

	model	country	reloc_dist	rank_least_moved
	Qwen/Qwen3.6-Plus	TWN	0.054	1
	Qwen/Qwen3.6-Plus	SGP	0.056	2
	Qwen/Qwen3.6-Plus	USA	0.111	3
	Qwen/Qwen3.6-Plus	URY	0.129	4
	Qwen/Qwen3.6-Plus	ARG	0.160	5
	claude-opus-4-8	ARG	0.073	1
	claude-opus-4-8	URY	0.085	2
	claude-opus-4-8	TWN	0.121	3
	claude-opus-4-8	CHL	0.143	4
	claude-opus-4-8	THA	0.170	5
	gemini-3.6-flash	URY	0.051	1
	gemini-3.6-flash	USA	0.080	2
	gemini-3.6-flash	ARG	0.199	3
	gemini-3.6-flash	TWN	0.202	4
	gemini-3.6-flash	CHL	0.241	5
	gpt-5.5	ARG	0.073	1
	gpt-5.5	URY	0.074	2
	gpt-5.5	USA	0.102	3
	gpt-5.5	SGP	0.119	4
	gpt-5.5	CHL	0.133	5

```
cat("\n=== how many models put each country in its bottom five ===\n")
```

=== how many models put each country in its bottom five ===

```
ranked %>% filter(rank_least_moved <= 5) %>% count(country, name = "n_models") %>%  
  arrange(desc(n_models), country) %>% as.data.frame() %>% print(row.names = FALSE)
```

country	n_models
ARG	4
URY	4
CHL	3
TWN	3
USA	3
SGP	2
THA	1

```
cat("\n=== relocation by country, averaged over models (ten smallest) ===\n")
```

=== relocation by country, averaged over models (ten smallest) ===

```
ranked %>% group_by(country) %>%  
  summarise(mean_reloc = round(mean(reloc_dist), 3),  
            min_reloc = round(min(reloc_dist), 3), .groups = "drop") %>%  
  arrange(mean_reloc) %>% head(10) %>% as.data.frame() %>% print(row.names = FALSE)
```

country	mean_reloc	min_reloc
URY	0.085	0.051
USA	0.120	0.080
ARG	0.126	0.073
TWN	0.131	0.054
CHL	0.174	0.133
SGP	0.182	0.056
CYP	0.238	0.165
KOR	0.262	0.185
JPN	0.277	0.182
HKG	0.295	0.188

```
cat(sprintf("\nsmallest single relocation: %.3f | mean: %.3f | median: %.3f\n",
          min(ranked$reloc_dist), mean(ranked$reloc_dist), median(ranked$reloc_dist)))
```

smallest single relocation: 0.051 | mean: 0.468 | median: 0.429

```
saveRDS(list(reloc = reloc, ranked = ranked, boot_base = b_base,
            boot_reloc = b_reloc, boot_improve = b_impr), "c0_relocation.rds")
```

```
#' ## Sensitivity: shared twelve-item base
```

```
#'
```

```
#' Rescores C1 without G006 so both conditions rest on the same components, and
#' recomputes the relocation. If the ordering and magnitude survive, the
#' relocation result is not an artefact of the missing item.
```

```
vals <- readRDS("model_item_values.rds")
```

```
params <- read_csv("standardization_params.csv", show_col_types = FALSE)
```

```
base <- read_csv("item_base_by_country.csv", show_col_types = FALSE)
```

```
coords_12 <- vals %>%
```

```
  filter(item != "G006", condition %in% c("C0", "C1")) %>%
```

```
  left_join(params, by = "item") %>%
```

```
  left_join(base, by = c("country" = "iso", "item")) %>%
```

```
  filter(keep) %>%
```

```
  mutate(z = sign * (value - mu) / sd) %>%
```

```
  group_by(model, country, condition, dim) %>%
```

```
  summarise(score = mean(z), .groups = "drop") %>%
```

```
  pivot_wider(names_from = dim, values_from = score)
```

```
c0_12 <- coords_12 %>% filter(condition == "C0") %>%
```

```
  group_by(model) %>% summarise(c0_1 = mean(Dim1), c0_2 = mean(Dim2), .groups = "drop")
```

```
reloc_12 <- coords_12 %>% filter(condition == "C1") %>%
```

```
  transmute(model, country = as.character(country), c1_1 = Dim1, c1_2 = Dim2) %>%
```

```
  inner_join(c0_12, by = "model") %>%
```

```
  mutate(reloc_dist = sqrt((c1_1 - c0_1)^2 + (c1_2 - c0_2)^2))
```

```
cat("\n=== sensitivity: relocation on the shared 12-item base (G006 dropped from both) ===\n")
```

=== sensitivity: relocation on the shared 12-item base (G006 dropped from both) ===

```
report_ci(boot_paired(reloc_12, "reloc_dist"), "relocation, 12-item base, 95% CI")
```

```
=== relocation, 12-item base, 95% CI ===  
          X2.5.  X50. X97.5. vs_ref  
Qwen/Qwen3.6-Plus 0.376 0.458  0.543  ABOVE  
claude-opus-4-8   0.335 0.391  0.448  ABOVE  
gemini-3.6-flash  0.478 0.552  0.631  ABOVE  
gpt-5.5           0.415 0.489  0.570  ABOVE
```